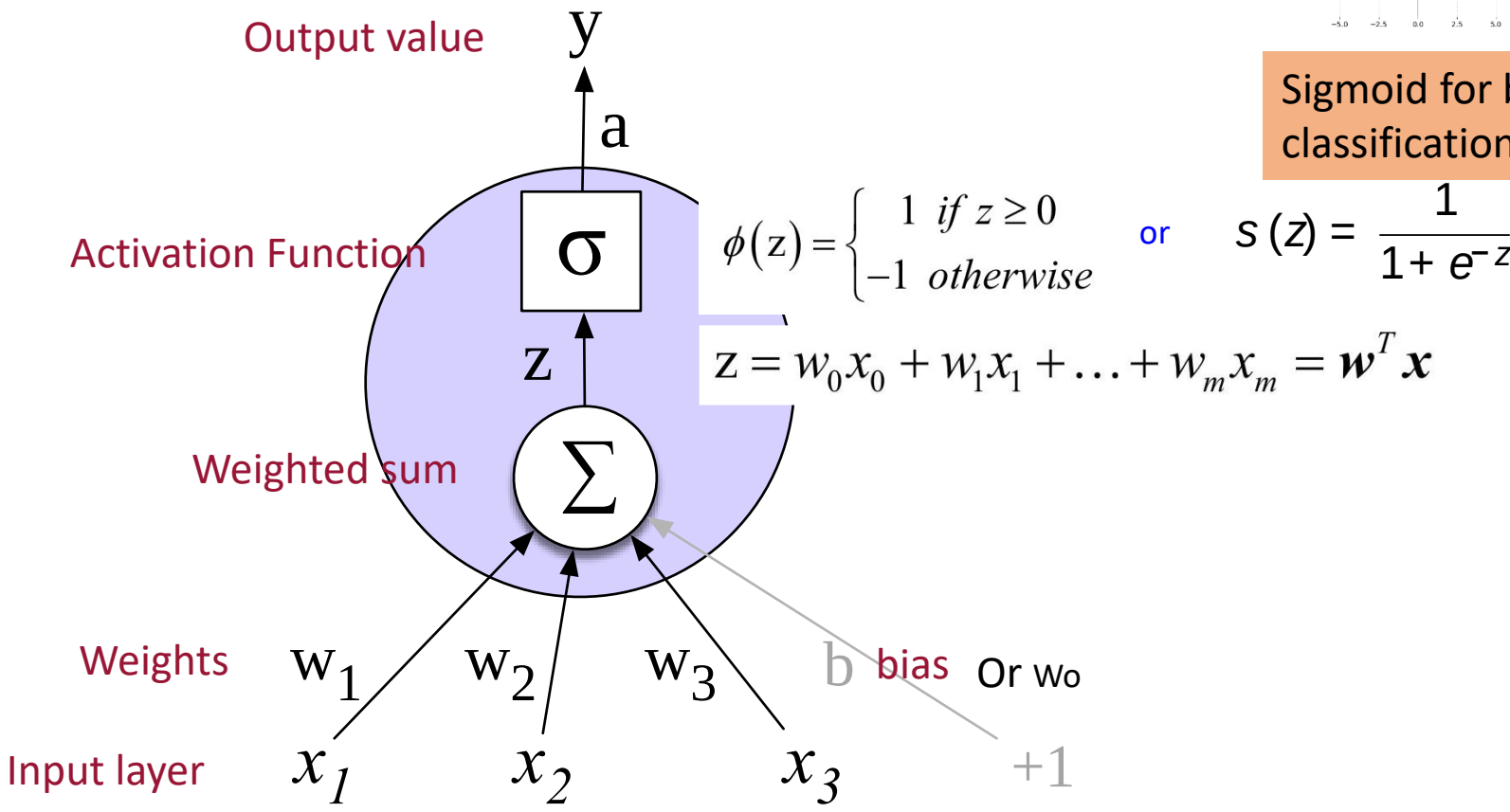
Activation Functions

Neural Network Unit

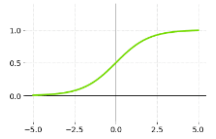
Perceptron



$$\phi(z) = \begin{cases} 1 \ \ if\ z \geq 0 \\ -1 \ \ otherwise \end{cases}$$

or

$$s(z) = \frac{1}{1+e^{-z}}$$



Sigmoid for binary classification

$$z = w_0 x_0 + w_1 x_1 + \ldots + w_m x_m = \boldsymbol{w}^T \boldsymbol{x}$$

Example

Suppose a unit has:

$w = [0.2, 0.3, 0.9]$

$b = 0.5$

What happens with input x:

$x = [0.5, 0.6, 0.1]$

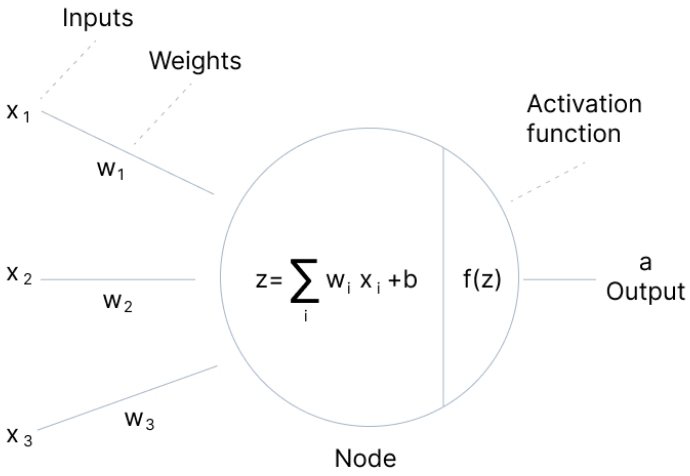
$$Z = w_1x_1 + w_2x_2 + w_3x_3 + b$$
$$= 0.2 \times 0.5 + 0.3 \times 0.6 + 0.9 \times 0.1 + 0.5$$
$$= 0.87$$

Output:
Using Step function
S(z)=0.7
Output= 1

If Sigmoid is taken with 0.5 as threshold
S(z) = 0.70
S(z)>=0.5 → 1

Activation functions

- An Activation Function decides whether a neuron should be activated (fired) or not. In other words, it transforms the weighted sum of the input to the desired form
- The activation function determines the output of the NN unit based on the weighted sum of the inputs and the bias term.
  - It Captures the presence of non-linear relationships between the inputs.
  - It contributes to the conversion of the input into a more usable output.
    - In other words it maps the resulting values in between 0 to 1 or -1 to 1 etc. (depending upon the chosen function)



V7 Labs

Why need activation function?

- **Activation function** is needed in a neural network to introduce **non-linearity** into the model. Without it, a neural network becomes just a linear model — no matter how many layers it has.
- Let us suppose, we have a neural network working *without* the activation functions.

- In that case, every neuron will only be performing a linear transformation on the inputs using the weights and biases. It is because it doesn't matter how many hidden layers we attach in the neural network; all layers will behave in the same way because the composition of two linear functions is a linear function itself.

- Although the neural network will become simpler, learning any complex task is impossible, and our model would be just a linear regression model.

Suppose a layer computes:

$$z = Wx + b$$

This is a **linear transformation**.

If you stack multiple layers **without activation**:

$$z_1 = W_1 x + b_1$$

$$z_2 = W_2 z_1 + b_2$$

Substituting:

$$z_2 = W_2(W_1 x + b_1) + b_2$$

$$z_2 = (W_2 W_1)x + (W_2 b_1 + b_2)$$

This is still of the form:

$$z_2 = W' x + b'$$

Types

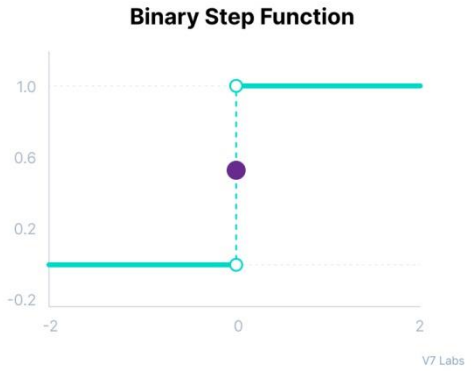
- Binary step Function
- Linear activation functions
- Non-linear activation functions

Activation Functions…

- Binary Step Function
  - Binary step function depends on a threshold value that decides whether a neuron should be activated or not.
    The input fed to the activation function is compared to a certain threshold; if the input is greater than it, then the neuron is activated, else it is deactivated, meaning that its output is not passed on to the next hidden layer.
  - Mathematically it can be represented as:

$$\textit{Binary step}$$

$$f(x) = \begin{cases} 0 & for\ x < 0 \\ 1 & for\ x \geqslant 0 \end{cases}$$

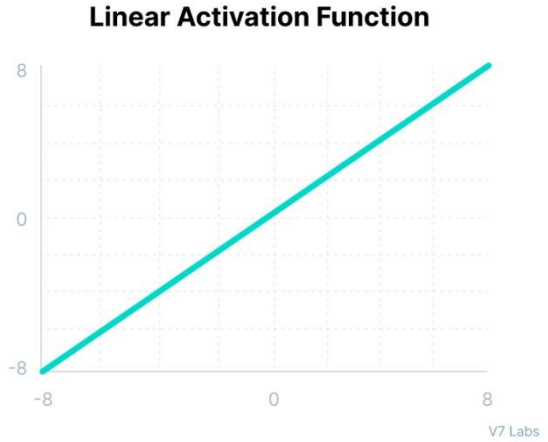


Limitations of binary step function:

- It cannot provide multi-value outputs—for example, it cannot be used for multi-class classification problems.
- The gradient of the step function is zero, which causes a hindrance in the backpropagation process.

Linear Activation Function

The linear activation function, also known as "no activation," or "identity function" (multiplied x1.0), is where the activation is proportional to the input.
The function doesn't do anything to the weighted sum of the input, it simply spits out the value it was given.



Mathematically it can be represented as:

$$f(x) = x$$

A linear activation function has major problems :

- All layers of the neural network will collapse into one if a linear activation function is used. No matter the number of layers in the neural network, the last layer will still be a linear function of the first layer. So, essentially, a linear activation function turns the neural network into just one layer.

Then how do we train regression models?
For regression we typically use:
Hidden layers → Non-linear activation (ReLU, tanh, etc.)
Output layer → Linear activation

Non-Linear Activation Functions

The linear activation function discussed so far is simply a linear regression model.

Because of its limited power, this does not allow the model to create complex mappings between the network's inputs and outputs.

Non-linear Activation Functions

Non-linear activation functions allow the network to learn **non-linear mappings** because their derivatives depend on the neuron's input value.

Example: Sigmoid

$$f(z) = \frac{1}{1 + e^{-z}}$$

$$f'(z) = f(z)(1 - f(z))$$

Now the gradient depends on:

- the input
- the neuron activation
- the current state of the network

This allows **different neurons to update differently**, enabling complex learning.

They allow the stacking of multiple layers of neurons as the output would now be a non-linear combination of input passed through multiple layers. Any output can be represented as a functional computation in a neural network.

- The Nonlinear Activation Functions are mainly divided on the basis of their **range or curves**-

1. Sigmoid or Logistic Activation Function

The **sigmoid function** (also called logistic function) is:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

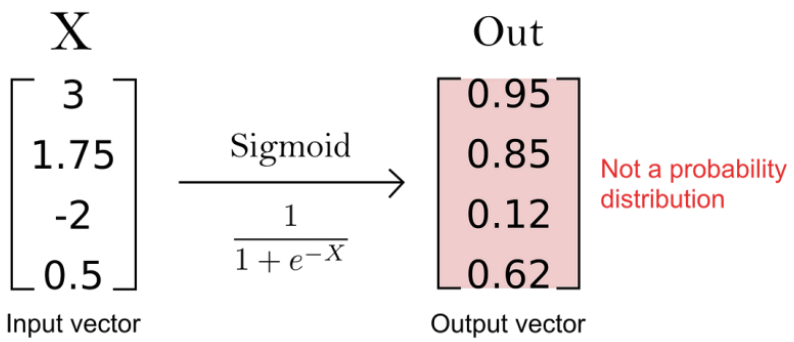
It maps any real number:

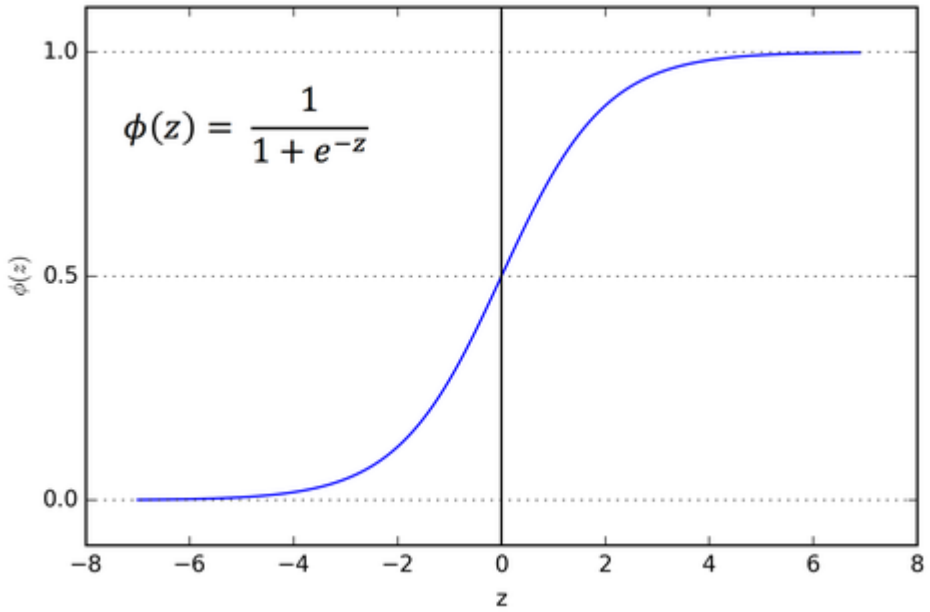
$$(-\infty, +\infty)$$

to:

$$(0, 1)$$

The Sigmoid Function curve looks like a S-shape.





Sigmoid became popular because:

- It outputs values between **0 and 1**
- It can represent **probability**
- It is smooth and differentiable everywhere

That is why it is used in:

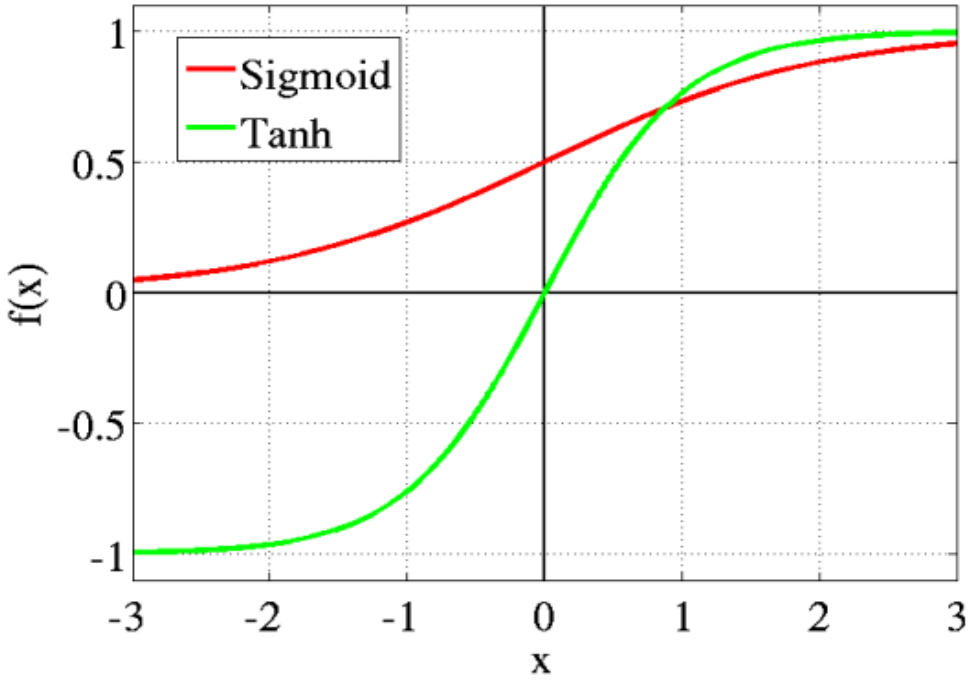
- Binary classification
- Logistic regression
- Output layer of binary neural networks

2. Tanh or hyperbolic tangent Activation Function

- Tanh is also like logistic sigmoid
- The range of the tanh function is from (-1 to 1).
- The **tanh activation function** is:

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

- The advantage is that the negative inputs will be mapped strongly negative and the zero inputs will be mapped near zero in the tanh graph.



3. ReLU (Rectified Linear Unit) Activation Function

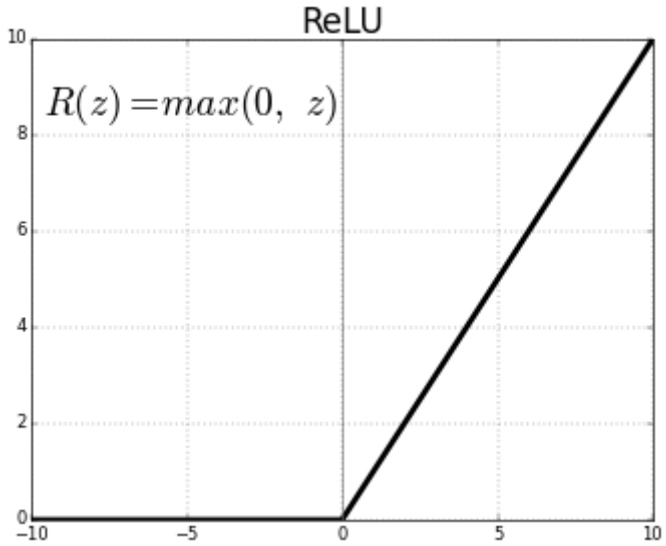
- The ReLU is one of the most used activation function in the world right now. Itis used in almost all the convolutional neural networks or deep learning.

The **ReLU activation function** is defined as:

$$f(z) = \max(0, z)$$

This means:

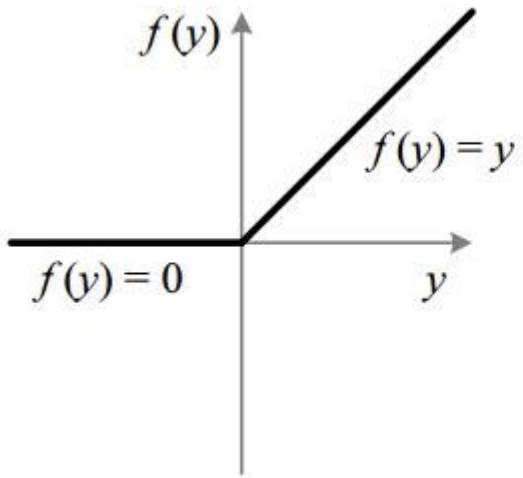
- If $z > 0 \rightarrow$ output = z
- If $z \leq 0 \rightarrow$ output = 0

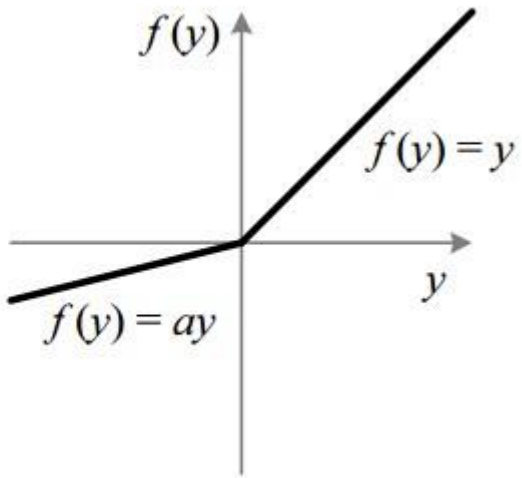


- But the issue is that all the negative values become zero immediately which decreases the ability of the model to fit or train from the data properly.
- That means any negative input given to the ReLU activation function turns the value into zero immediately in the graph, which in turns affects the resulting graph by not mapping the negative values appropriately.

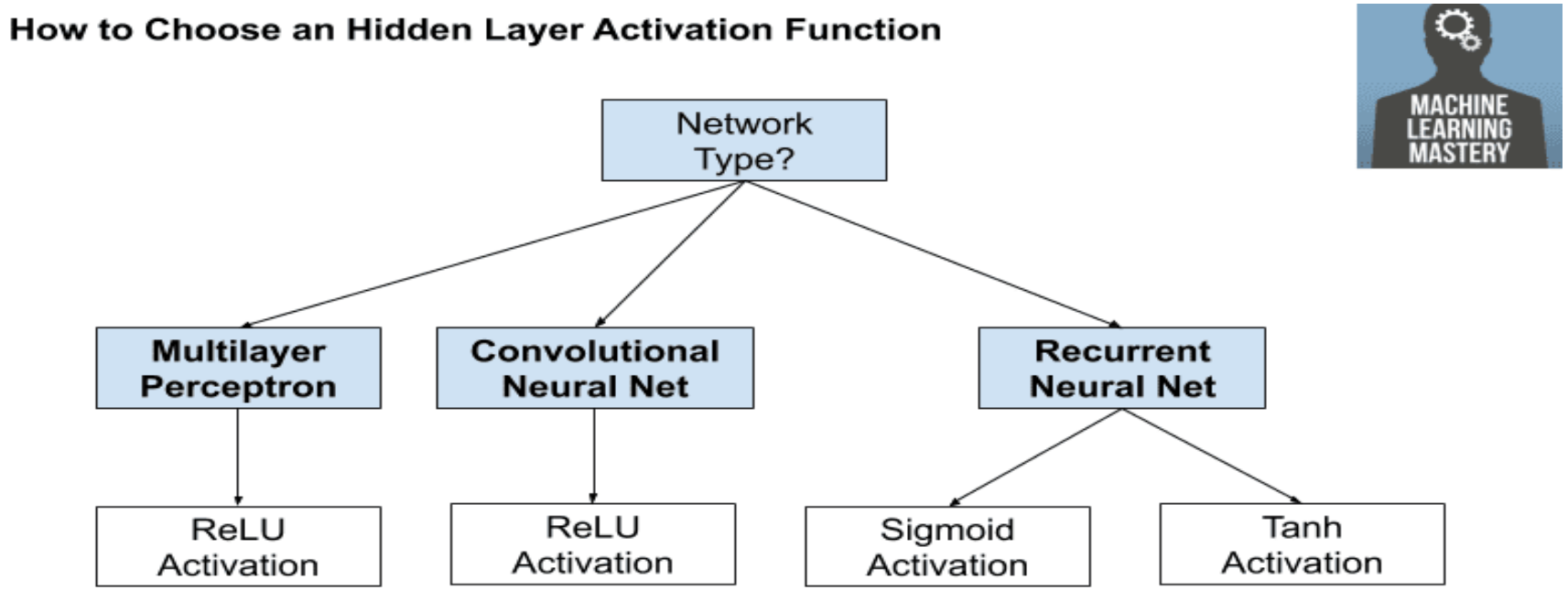
4. Leaky ReLU

- It is an attempt to solve the dying ReLU problem
- The leak helps to increase the range of the ReLU function. Usually, the value of **a** is 0.01 or so.
- When **a is not 0.01** then it is called **Randomized ReLU**





- Hidden layer activation functions:
  - Rectified Linear Activation (**ReLU**)
  - Logistic (**Sigmoid**)
  - Hyperbolic Tangent (**Tanh**)

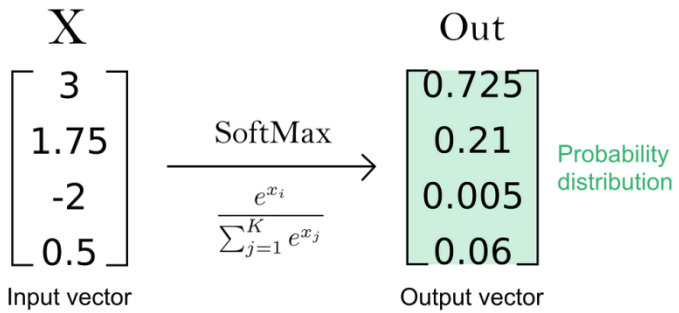


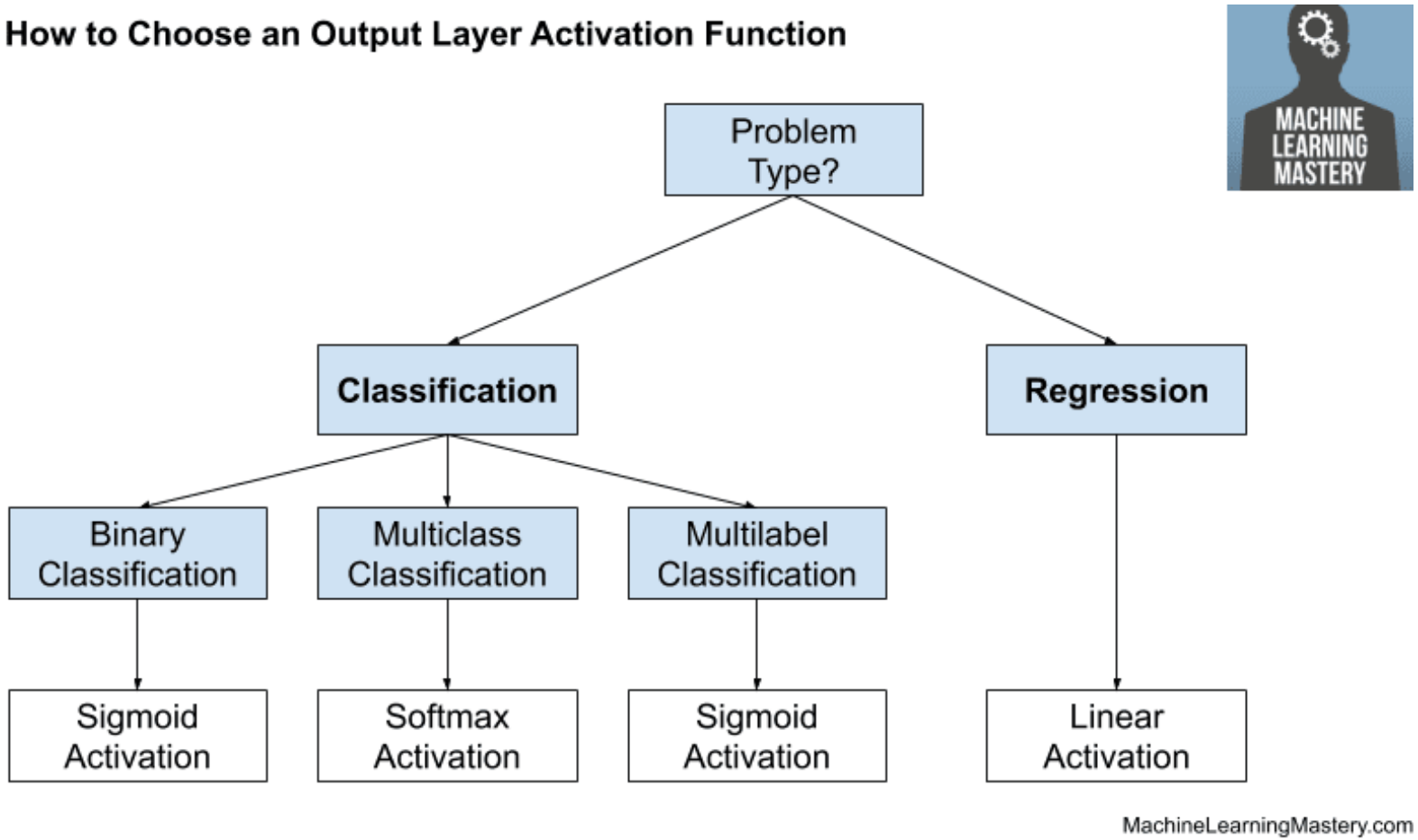
- **Activation functions for Output Layers**
  - Linear activation function: for regression problems
  - Logistic (Sigmoid)
  - Softmax

- **Softmax**
  - For **Binary classification**, both **sigmoid**, as well as **softmax**, are equally approachable but in case of multi-class classification problem we generally use softmax
  - The softmax function is calculated as follows:

$$\text{Softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}$$

  - The softmax function outputs a vector of values that sum to 1.0 that can be interpreted as probabilities of class membership.



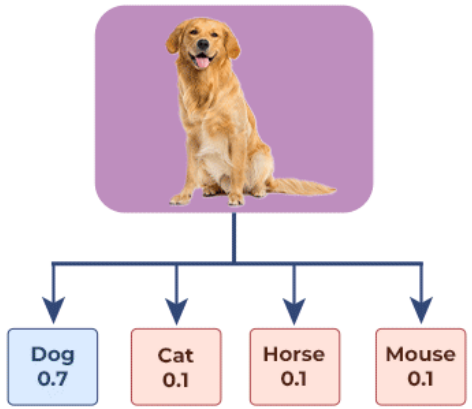


Multiclass classification :
The goal is to assign instances to one of multiple predefined classes or categories, where each instance belongs to exactly one class.

Multilabel classification is a machine learning task where each instance can be associated with multiple labels simultaneously

Mutliclass Classification vs multilabel classification

Multiclass Classification

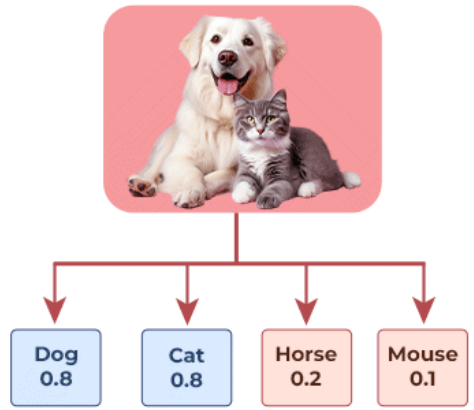


Classes

(pick one class)

- [x] Dog
- [ ] Cat
- [ ] Horse
- [ ] Mouse

Multilabel Classification



Classes

(pick all the labels present in the image)

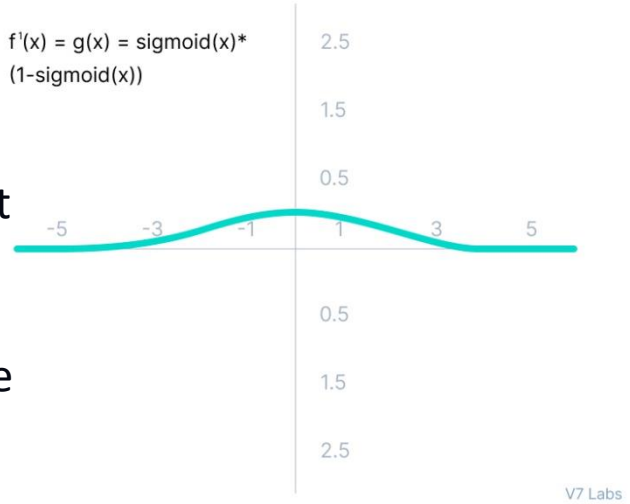
- [x] Dog
- [x] Cat
- [ ] Horse
- [ ] Mouse

Appendix

sigmoid/logistic activation function is one of the most widely used functions:

•It is commonly used for models where we have to predict the probability as an output. Since probability of anything exists only between the range of 0 and 1, sigmoid is the right choice because of its range.
•The function is differentiable and provides a smooth gradient, i.e., preventing jumps in output values. This is represented by an S-shape of the sigmoid activation function.

The limitations of sigmoid function are discussed below:



•The derivative of the function is f'(x) = sigmoid(x)*(1-sigmoid(x)).

As we can see from the above Figure, the gradient values are only significant for range -3 to 3, and the graph gets much flatter in other regions.

It implies that for values greater than 3 or less than -3, the function will have very small gradients. As the gradient value approaches zero, the network ceases to learn and suffers from the *Vanishing gradient* problem.

•The output of the logistic function is not symmetric around zero. So the output of all the neurons will be of the same sign. This makes the training of the neural network more difficult and unstable.

https://www.v7labs.com/blog/neural-networks-activation-functions#:~:text=drive%20V7%27s%20tools.-,What%20is%20a%20Neural%20Network%20Activation%20Function%3F,prediction%20using%20simpler%20mathematical%20operations.